Planning Détaillé du Projet
Catégorisation Contrastive et Pertinence

Groupe : Samy, Issa, Anass, Hussein

February 21, 2025

Introduction

Ce projet vise à mettre en œuvre la *catégorisation contrastive* et la *pertinence*, en s'appuyant sur l'idée d'un **opérateur de contraste** (Dessalles, Sileno et al.). Il s'agit de représenter les données (ex. chevaux, licornes, tomates, visages rougissants) dans un espace conceptuel vectoriel et d'appliquer le *contraste* pour faire ressortir les différences saillantes. La *pertinence* s'obtient en se focalisant sur la dimension où la différence est la plus grande, et en reliant le vecteur de différence à un prototype "nommable" (p. ex. "rouge", "vert", "cornu") : C=O-P.

Organisation du Planning

Le projet se déroule sur **8 phases** de travail régulières sur des séances dédiés et de travail personnelles, plus un *sprint final* (dernière semaine d'Artishow). Ci-dessous, on détaille pour chaque semaine : l'objectif, les tâches, et la répartition par membre (A, B, C, D). Nous nous repartirons en fonction des goûts de chacun sur les tâches.

Notation :

- Contraste = $\mathbf{C} = \frac{\mathbf{o-p}}{\rho}$: opérateur principal.
- Merging (inverse) : combiner un prédicat (p. ex. "rouge") à un concept de référence (p. ex. "brique").
- Force d'appartenance : $strength(r) = \frac{|\mathbf{C} \cap M(r)|}{|\mathbf{C}|}$, etc.
- Pondération, hiérarchisation : prise en compte de la saillance et des dimensions multiples.

1 Planning par Séances

Phase 1 (jusqu'au 3 mars): Lancement & Cadrage

Objectifs :

- Préciser la question de recherche : Comment formaliser l'opérateur de contraste et la pertinence dans un espace conceptuel multidimensionnel ?
- Répartir les rôles et clarifier les livrables (prototype Python, rapport).

Tâches :

1. **(A)** : Synthétiser rapidement toute la bibliographie (Dessalles, Sileno, etc.).
2. **(B)** : Définir la structure de l'espace conceptuel (dimensions *vs.* catégories).
3. **(C)** : Proposer un plan pour la génération du **jeu de données** (données synthétiques, ex. licornes, tomates, etc.).
4. **(D)** : Installer le repo Python, planifier l'architecture de code (un module `contrast.py`, un module `merge.py`, etc.).

Phase 2 (séance 10 et 18 mars) : Approfondissement Théorique (Contraste, Pertinence)

Objectifs :

- Plonger dans les formules : $\mathbf{C} = \frac{\mathbf{o}-\mathbf{p}}{\rho}$, s'assurer de la *normalisation*, la *blanchiment* (*whitening*).
- Discuter du **seuil de saillance** : à partir de quelle valeur absolue juge-t-on la différence notable ?

Tâches :

1. **(A)** : Formaliser le *contrast operator* en multidimension : comment traiter la *pondération* si certaines dimensions sont plus importantes ?
2. **(B)** : Proposer un *algorithme* : `contrast(o, p, sigma, rho)` $\rightarrow$ `C`.
3. **(C)** : Discuter la *force d'appartenance* à une catégorie r : $strength(r) = |C \cap M(r)|/|C|$. Mettre en place un petit exemple unidimensionnel.
4. **(D)** : Commencer à coder un **prototype minimal** en Python (uni-dimension) pour tester la force d'appartenance.

Phase 3 (25 mars et 1 avril) : Multidimension & Calibration

Objectifs :

- Généraliser à n dimensions : $\mathbf{C} = (C_1, \ldots, C_n)$.
- Réfléchir à la *calibration* de la pondération (certaines dimensions plus critiques).
- Introduire la *hiérarchisation* des catégories (ex. `fruit` $>$ `tomate`).

Tâches :

1. **(A)** : Implémenter la version multi-dimension dans `contrast.py` (avec un vecteur σ, ρ).
2. **(B)** : Gérer la *pondération* w_i par dimension : $C_i = w_i \times \frac{o_i - p_i}{\rho_i}$.
3. **(C)** : Définir un *seuil de saillance* adaptatif : ex. $|C_i| > \theta_i$ ou un ratio global.
4. **(D)** : Créer un mini-"dataset" (JSON ou CSV) de points en 2D ou 3D (ex. `Hue`, `Roundness`, etc.) pour tester.

Phase 4 (8 et 15 avril) : Génération de Dataset & Exemples Concrets

Objectifs :

- Créer un **jeu de données synthétique** (licornes, chevaux, tomates, ou visages., ce qui serait plus compliqué).
- Implémenter un *exemple* de *Left-of* en vision (ou un cas de position) pour tester la notion de *contraste spatial*.

Tâches :

1. **(A)** : Générer $N = 100$ points 2D (ex. `(color, size)`) regroupés en 2–3 catégories (cheval, licorne, visages, livres fin etc).
2. **(B)** : Ajouter des points *anormaux* (licornes noires, etc.) pour tester la *détection d'anomalies*.
3. **(C)** : Illustrer un *Left-of* en 2D : contraster l'objet A par rapport à B, agréger (A $\ominus$ B), etc.
4. **(D)** : Mettre à jour le code `contrast.py` pour gérer la dimension spatiale ("Left-of").

Phase 5 (16 avril et 5 mai) : Merging (Inverse du Contraste) & Tests Unitaires

Objectifs :

- Implémenter la *fusion* (`merge`) : brique + rouge $\rightarrow$ brique rouge.
- Réaliser des **tests unitaires** sur `contrast` et `merge`.

Tâches :

1. **(A)** : Formaliser la fusion : `merge(contraste, reference)` $\rightarrow$ `nouveau prototype` (ou attribut modifié).
2. **(B)** : Exemples : licorne = cheval + corne, tomate noire = tomate + noir.
3. **(C)** : Créer des **tests** dans `test_contrast.py` et `test_merge.py`.
4. **(D)** : Vérifier la cohérence (catégorie résultante, distances, etc.).

Phase 6 (12 et 19 mai): Approfondissement - Clustering Incrémental, Anomalies

Objectifs :

- Exploiter le *contraste* pour du **clustering incrémental sans paramètres**.
- Détection d'anomalies à partir du vecteur de contraste (distance trop grande).

Tâches :

1. **(A)** : Proposer un algorithme de *clustering incrémental* : on part d'un prototype, si un nouvel objet s'écarte trop, on crée un nouveau cluster.
2. **(B)** : Implémenter la *détection d'anomalies* : $\|\mathbf{C}\|$ ou $\max_i |C_i| > \textit{seuil} \Rightarrow$ anomalie.
3. **(C)** : Tester sur le dataset de la Séance 4 (tomates noires, licornes).
4. **(D)** : Documenter la fonction `detect_anomaly(o, p, rho)` dans le code.

Phase 7 (26 mai et 2 juin) : Calibration Finale & Validation

Objectifs :

- Calibrer la *pondération* w_i, hiérarchiser les catégories (ex. fruit > tomate).
- Valider les résultats (précision, qualité de la pertinence).

Tâches :

1. **(A)** : Mettre en place un *module* `calibration.py` qui permet d'ajuster w_i en fonction de la variance dans la dimension i.
2. **(B)** : Hiérarchiser : si on *rate* la catégorie `tomate`, on tente `fruit`.
3. **(C)** : Mesurer la qualité : ratio de bonnes descriptions, qualité de la détection d'anomalies (F1-score).
4. **(D)** : Rédiger un mini-rapport de validation (tableaux de résultats).

Phase 8 (Dernière semaine 23-28 juin) : Sprint Final, Préparation Soutenance, et approfondissement

Objectifs :

- Finaliser le **prototype Python** et la **démo**.
- Rédiger le **rapport** et les **slides**.

Tâches :

1. **(A)** : Mise en forme finale du code (`contrast.py`, `merge.py`, `calibration.py`, `test_*.py`).
2. **(B)** : Diaporama de présentation (expliquer l'opérateur de contraste, exemples licorne, tomate noire, etc.).
3. **(C)** : Démo interactive (montrer comment le système catégorise, signale anomalies, etc.).
4. **(D)** : Rapport final : cadre théorique (Dessalles, Sileno), architecture, tests.

Entre les séances, le but sera de finir le travail nécessaire pour atteindre les objectifs fixés.

Conclusion

Voici donc les étapes principales qui constituent le projet :

- Définition de l'opérateur de *contraste* et de la *pertinence*,
- Passage à la *multidimension* (pondération, seuil de saillance),
- Génération de dataset,
- Implémentation de l'opération *inverse* (merge),
- Clustering incrémental, détection d'anomalies,
- Calibration et validation,
- Sprint final (démo, rapport).

L'objectif final est de montrer comment la **catégorisation contrastive** permet de générer des *descriptions pertinentes* (ex. "cheval avec une corne", "tomate noire", "visage plus rouge"), et d'étendre cette logique à des scénarios de *clustering incrémental* et de *détection d'anomalies*. Le tout s'appuie sur la *notion de vecteurs de différences* (contraste), enrichie par des mécanismes de *pondération* et de *hiérarchisation*.